

Chương 2 – Dự án Machine Learning từ đầu đến cuối

Notebook này chứa tất cả các mã mẫu và giải pháp cho các bài tập trong chương 2.

 [Open in Colab](#)

 [Open in Kaggle](#)

```
print("Welcome to Machine Learning!")
```

```
Welcome to Machine Learning!
```

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

✓ Lấy dữ liệu

Chào mừng bạn đến với Machine Learning Housing Corp.! Nhiệm vụ của bạn là dự đoán giá nhà trung bình tại các khu vực ở California, dựa trên một số đặc trưng từ các khu vực này.

✓ Tải dữ liệu

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
```

```

if not tarball_path.is_file():
    Path("datasets").mkdir(parents=True, exist_ok=True)
    url = "https://github.com/ageron/data/raw/main/housing.tgz"
    urllib.request.urlretrieve(url, tarball_path)
with tarfile.open(tarball_path) as housing_tarball:
    housing_tarball.extractall(path="datasets")
return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()

```

✓ Xem nhanh cấu trúc dữ liệu

housing.head()

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	3141
1	-122.22	37.86	21.0	7099.0	1106.0	24651
2	-122.24	37.85	52.0	1467.0	190.0	4983
3	-122.25	37.85	52.0	1274.0	235.0	5653
4	-122.25	37.85	52.0	1627.0	280.0	5663

housing.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude             20640 non-null  float64
1   latitude              20640 non-null  float64
2   housing_median_age    20640 non-null  float64
3   total_rooms           20640 non-null  float64
4   total_bedrooms        20433 non-null  float64
5   population            20640 non-null  float64
6   households            20640 non-null  float64
7   median_income         20640 non-null  float64
8   median_house_value    20640 non-null  float64
9   ocean_proximity       20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB

```

housing["ocean_proximity"].value_counts()

<1H OCEAN	9136
INLAND	6551

```
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND        5
Name: ocean_proximity, dtype: int64
```

```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.0000
mean	-119.569704	35.631861	28.639486	2635.763081	537.8705
std	2.003532	2.135952	12.585558	2181.615252	421.3850
min	-124.350000	32.540000	1.000000	2.000000	1.0000
25%	-121.800000	33.930000	18.000000	1447.750000	296.0000
50%	-118.490000	34.260000	29.000000	2127.000000	435.0000
75%	-118.010000	37.710000	37.000000	3148.000000	647.0000
max	-114.310000	41.950000	52.000000	39320.000000	6445.0000

Ô mã nguồn sau đây không xuất hiện trong sách. Nó tạo thư mục

`images/end_to_end_project` (nếu chưa tồn tại) và định nghĩa hàm `save_fig()` được sử dụng xuyên suốt notebook này để lưu các hình ảnh với độ phân giải cao cho sách.

```
# extra code - code to save the figures as high-res PNGs for the book

IMAGES_PATH = Path() / "images" / "end_to_end_project"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

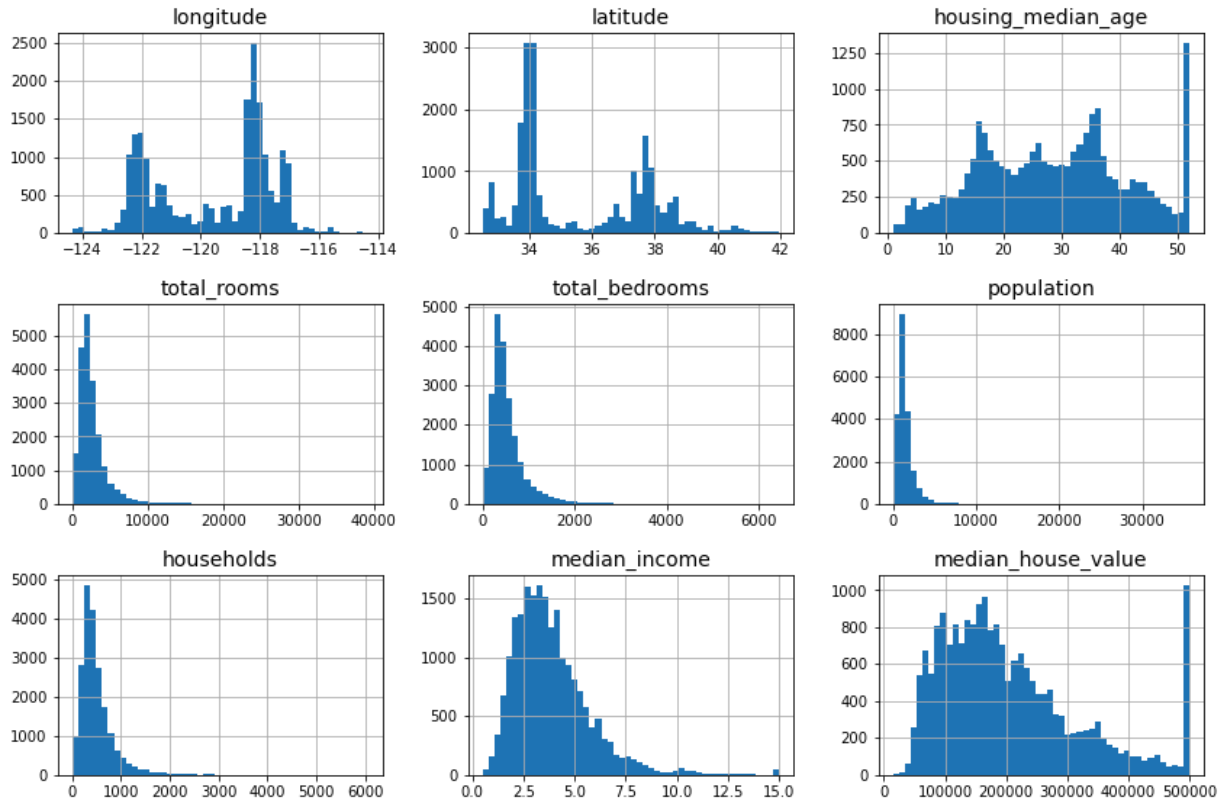
def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

```
import matplotlib.pyplot as plt

# extra code - the next 5 lines define the default font sizes
plt.rc('font', size=14)
plt.rc('axes', labelsizes=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsizes=10)
plt.rc('ytick', labelsizes=10)

housing.hist(bins=50, figsize=(12, 8))
```

```
save_fig("attribute_histogram_plots") # extra code
plt.show()
```



✓ Tạo tập kiểm tra (Test Set)

```
import numpy as np
```

```
def shuffle_and_split_data(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

```
train_set, test_set = shuffle_and_split_data(housing, 0.2)
len(train_set)
```

```
16512
```

```
len(test_set)
```

```
4128
```

Để đảm bảo rằng kết quả của notebook này không đổi mỗi khi chạy, chúng ta cần thiết lập seed ngẫu nhiên:

```
np.random.seed(42)
```

Đáng tiếc là điều này sẽ không đảm bảo notebook cho ra kết quả hoàn toàn giống hệt như trong sách, vì còn các nguồn biến đổi khác. Quan trọng nhất là việc các thuật toán được tinh chỉnh theo thời gian khi các thư viện phát triển. Vì vậy, hãy chấp nhận một số khác biệt nhỏ: hy vọng rằng hầu hết các đầu ra sẽ giống nhau, hoặc ít nhất là nằm trong phạm vi hợp lý.

Lưu ý: một nguồn ngẫu nhiên khác là thứ tự của các tập hợp (set) trong Python: nó dựa trên hàm `hash()`, hàm này được "muối" (salted) ngẫu nhiên khi Python khởi động. Để loại bỏ tính ngẫu nhiên này, giải pháp là đặt biến môi trường `PYTHONHASHSEED` thành `"0"` trước khi Python bắt đầu. Sẽ không có tác dụng nếu bạn thực hiện sau đó. May mắn thay, nếu bạn đang chạy notebook này trên Colab, biến này đã được thiết lập sẵn cho bạn.

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

```
housing_with_id = housing.reset_index() # adds an `index` column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

```
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)

test_set["total_bedrooms"].isnull().sum()
```

44

Để tìm xác suất một mẫu ngẫu nhiên gồm 1.000 người chứa ít hơn 48,5% nữ hoặc nhiều hơn 53,5% nữ khi tỷ lệ nữ trong quần thể là 51,1%, chúng ta sử dụng [phân phối nhị thức](#). Phương thức `cdf()` của phân phối nhị thức cho chúng ta xác suất số lượng nữ sẽ bằng hoặc ít hơn giá trị đã cho.

```
# extra code - shows how to compute the 10.7% proba of getting a bad sample

from scipy.stats import binom

sample_size = 1000
ratio_female = 0.511
proba_too_small = binom(sample_size, ratio_female).cdf(485 - 1)
proba_too_large = 1 - binom(sample_size, ratio_female).cdf(535)
print(proba_too_small + proba_too_large)
```

0.10736798530929946

Nếu bạn thích mô phỏng hơn là toán học, đây là cách bạn có thể nhận được kết quả xấp xỉ tương tự:

```
# extra code - shows another way to estimate the probability of bad sample

np.random.seed(42)

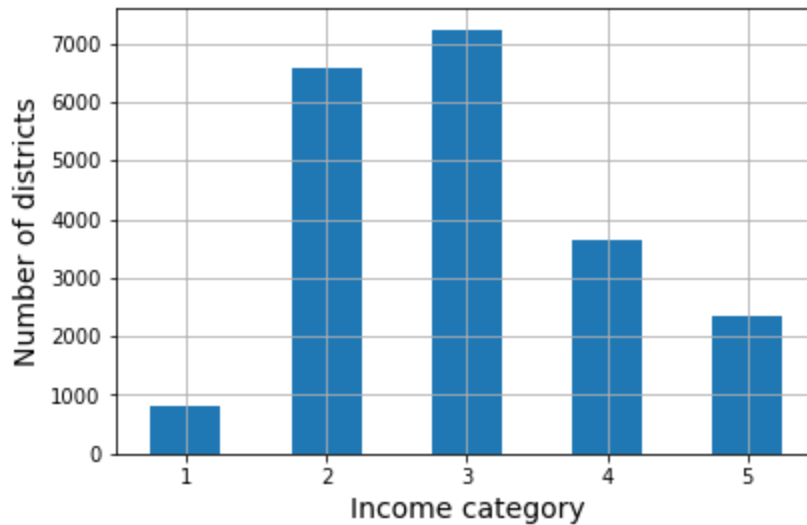
samples = (np.random.rand(100_000, sample_size) < ratio_female).sum(axis=1)
((samples < 485) | (samples > 535)).mean()
```

0.1071

```
housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                               labels=[1, 2, 3, 4, 5])
```

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
```

```
save_fig("housing_income_cat_bar_plot") # extra code
plt.show()
```



```
from sklearn.model_selection import StratifiedShuffleSplit

splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing, housing["income_cat"]):
    strat_train_set_n = housing.iloc[train_index]
    strat_test_set_n = housing.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

```
strat_train_set, strat_test_set = strat_splits[0]
```

Sẽ ngắn gọn hơn nhiều nếu chỉ lấy một lần phân tách phân tầng (stratified split):

```
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```

```
strat_test_set["income_cat"].value_counts() / len(strat_test_set)
```

```
3    0.350533
2    0.318798
4    0.176357
5    0.114341
1    0.039971
Name: income_cat, dtype: float64
```

```
# extra code - computes the data for Figure 2-10
```

```
def income_cat_proportions(data):
    return data["income_cat"].value_counts() / len(data)
```

```

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)

compare_props = pd.DataFrame({
    "Overall %": income_cat_proportions(housing),
    "Stratified %": income_cat_proportions(strat_test_set),
    "Random %": income_cat_proportions(test_set),
}).sort_index()
compare_props.index.name = "Income Category"
compare_props["Strat. Error %"] = (compare_props["Stratified %"] /
                                   compare_props["Overall %"] - 1)
compare_props["Rand. Error %"] = (compare_props["Random %"] /
                                   compare_props["Overall %"] - 1)
(compare_props * 100).round(2)

```

	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
Income Category					
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42

```

for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)

```

Khám phá và trực quan hóa dữ liệu để thu thập thông tin chi tiết

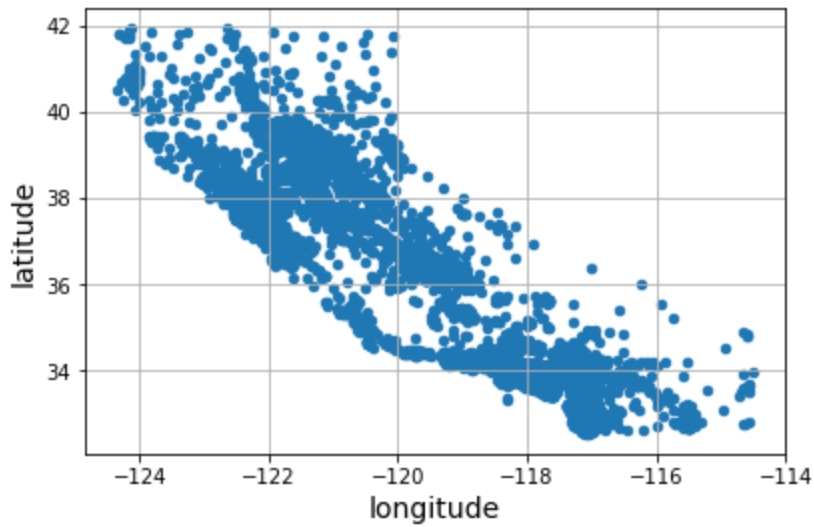
```
housing = strat_train_set.copy()
```

Trực quan hóa dữ liệu địa lý

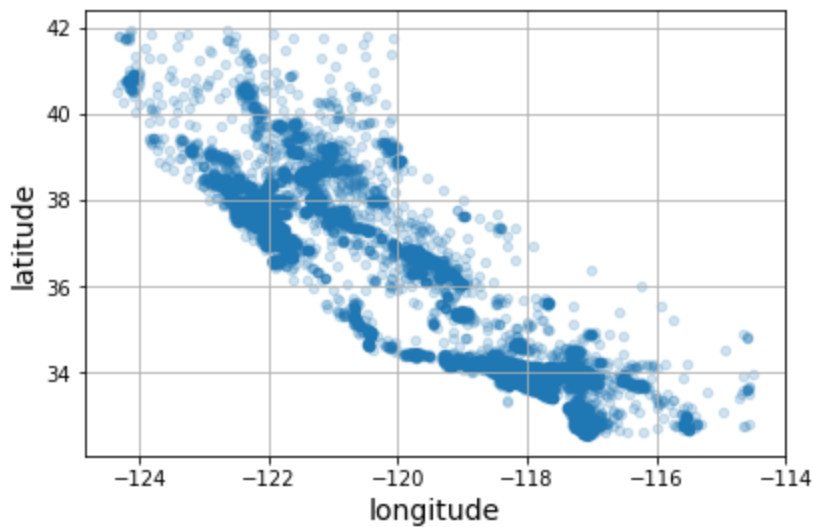
```

housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
save_fig("bad_visualization_plot") # extra code
plt.show()

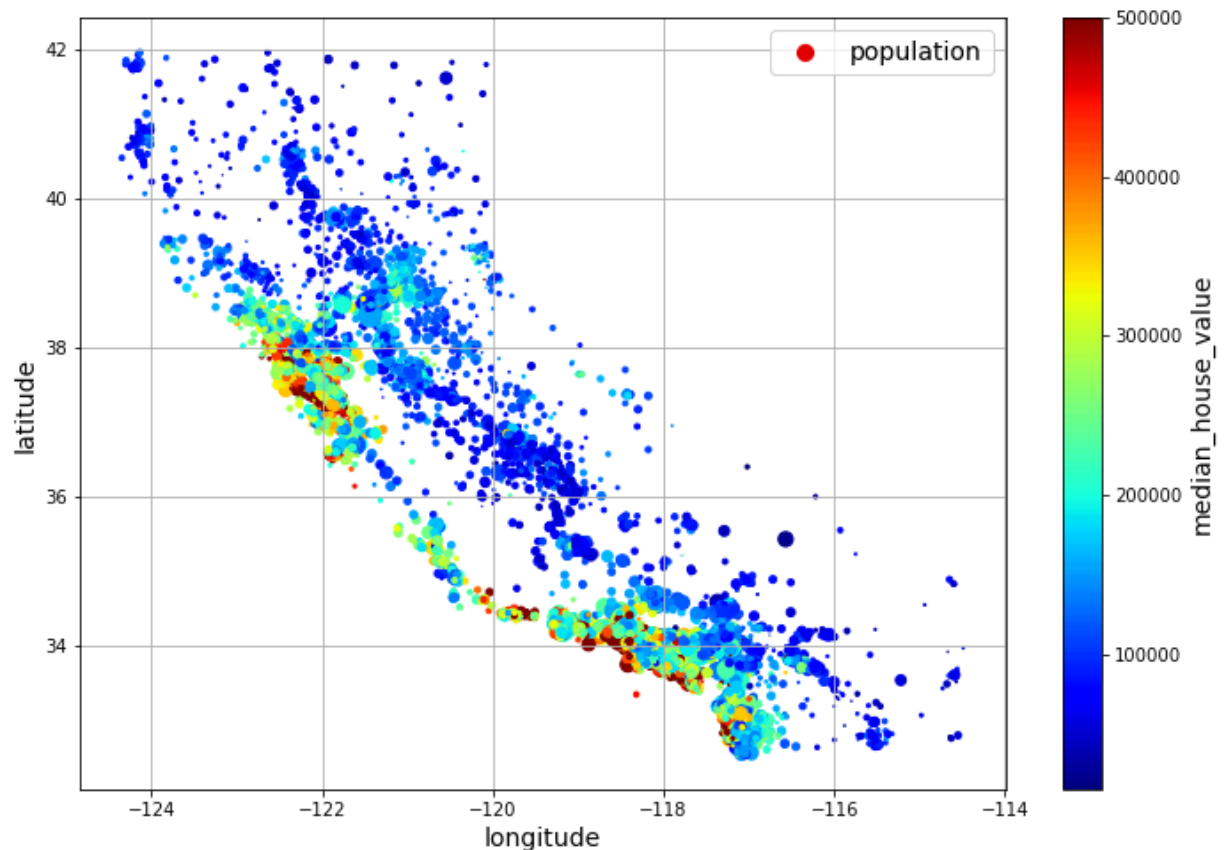
```

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
save_fig("better_visualization_plot") # extra code
plt.show()
```



```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
              s=housing["population"] / 100, label="population",
              c="median_house_value", cmap="jet", colorbar=True,
              legend=True, sharex=False, figsize=(10, 7))
save_fig("housing_prices_scatterplot") # extra code
plt.show()
```



Đổi số `sharex=False` sửa một lỗi hiển thị: nếu không có nó, các giá trị và nhãn trục x sẽ không được hiển thị (xem: <https://github.com/pandas-dev/pandas/issues/10611>).

Ô tiếp theo tạo ra hình ảnh đầu tiên trong chương (mã này không có trong sách). Đây chỉ là phiên bản được làm đẹp hơn của hình trước đó, với hình ảnh bản đồ California được thêm vào nền, tên nhãn đẹp hơn và không có lưới.

```
# extra code - this cell generates the first figure in the chapter

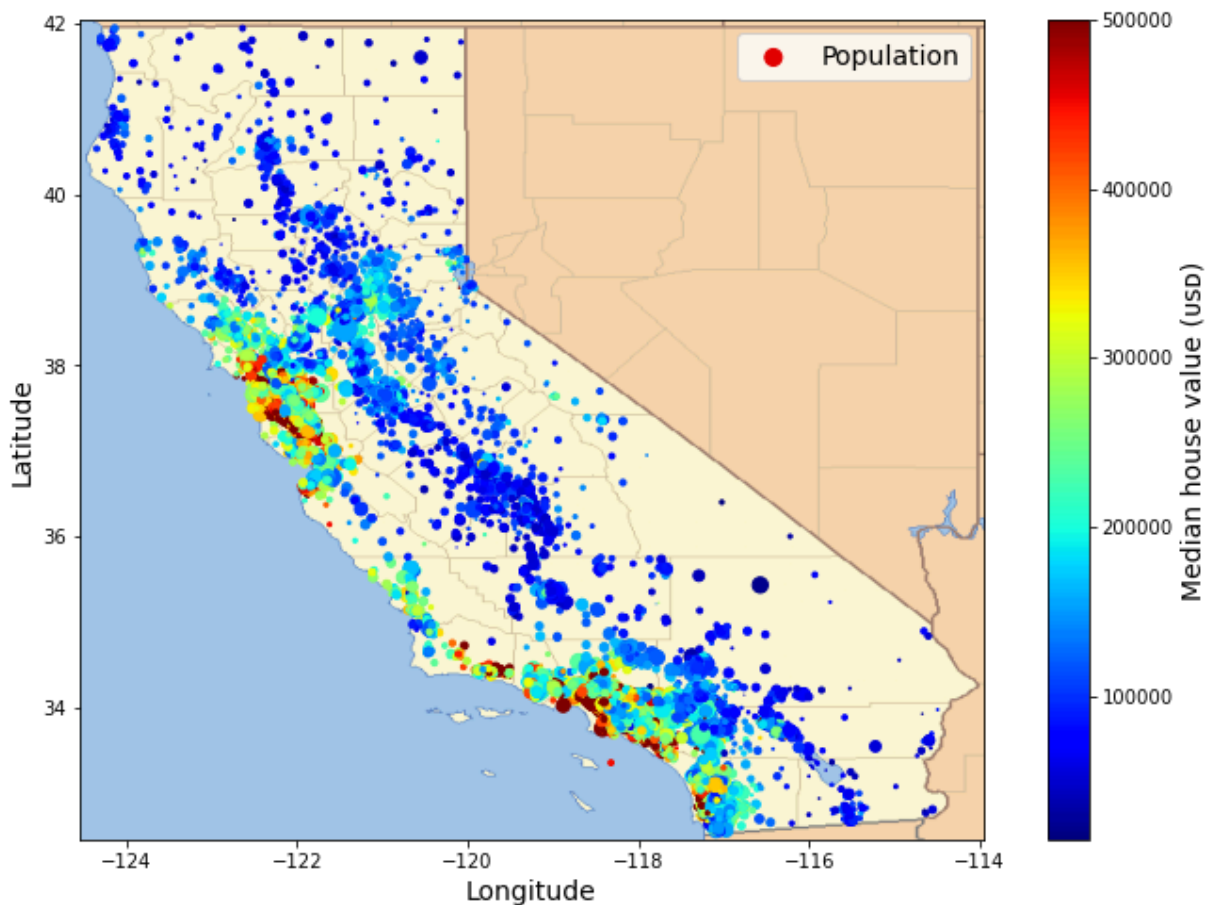
# Download the California image
filename = "california.png"
if not (IMAGES_PATH / filename).is_file():
    homl3_root = "https://github.com/ageron/handson-ml3/raw/main/"
    url = homl3_root + "images/end_to_end_project/" + filename
    print("Downloading", filename)
    urllib.request.urlretrieve(url, IMAGES_PATH / filename)

housing_renamed = housing.rename(columns={
    "latitude": "Latitude", "longitude": "Longitude",
    "population": "Population",
    "median_house_value": "Median house value (usd)"})
```

```
housing_renamed.plot(
    kind="scatter", x="Longitude", y="Latitude",
    s=housing_renamed["Population"] / 100, label="Population",
    c="Median house value (usd)", cmap="jet", colorbar=True,
    legend=True, sharex=False, figsize=(10, 7))

california_img = plt.imread(IMG_PATH / filename)
axis = [-124.55, -113.95, 32.45, 42.05]
plt.axis(axis)
plt.imshow(california_img, extent=axis)

save_fig("california_housing_prices_plot")
plt.show()
```



✓ Tìm kiếm các tương quan

Lưu ý: kể từ Pandas 2.0.0, đối số `numeric_only` mặc định là `False`, vì vậy chúng ta cần đặt nó thành `True` một cách rõ ràng để tránh lỗi.

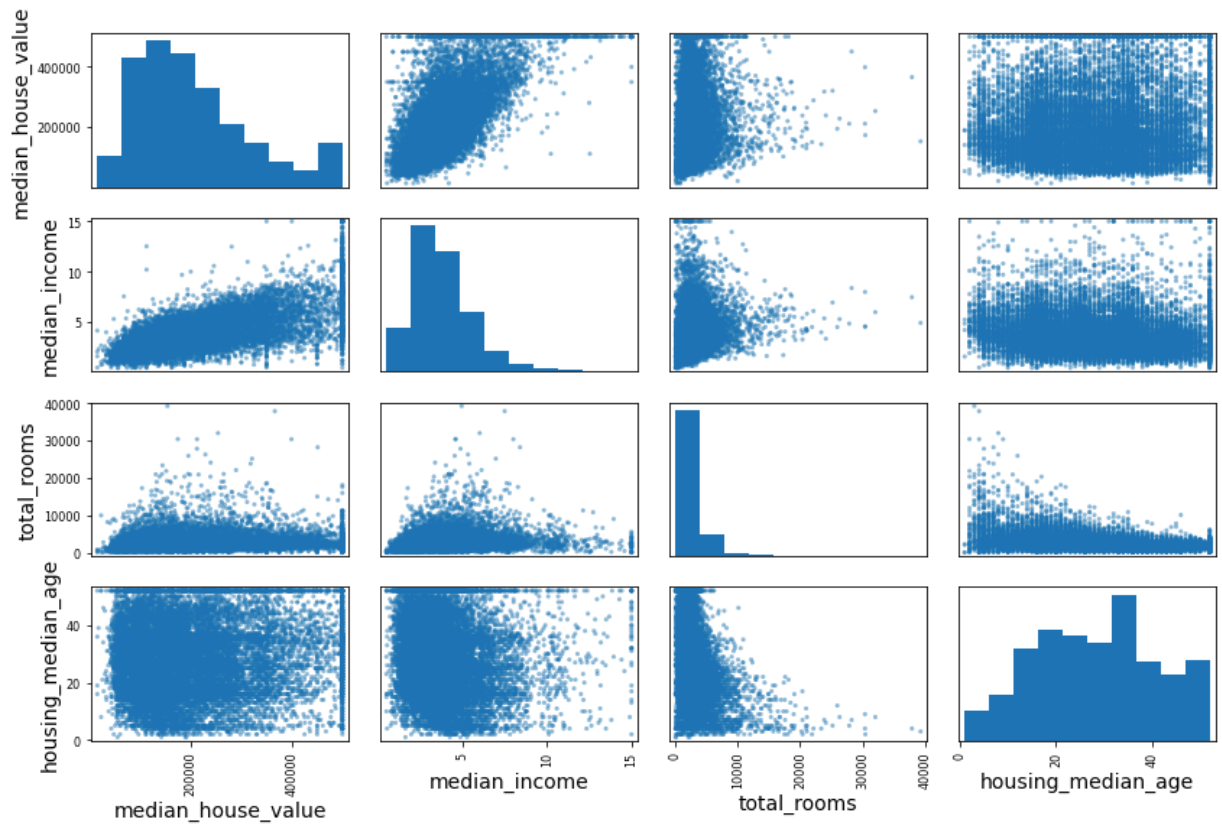
```
corr_matrix = housing.corr(numeric_only=True)
```

```
corr_matrix["median_house_value"].sort_values(ascending=False)
```

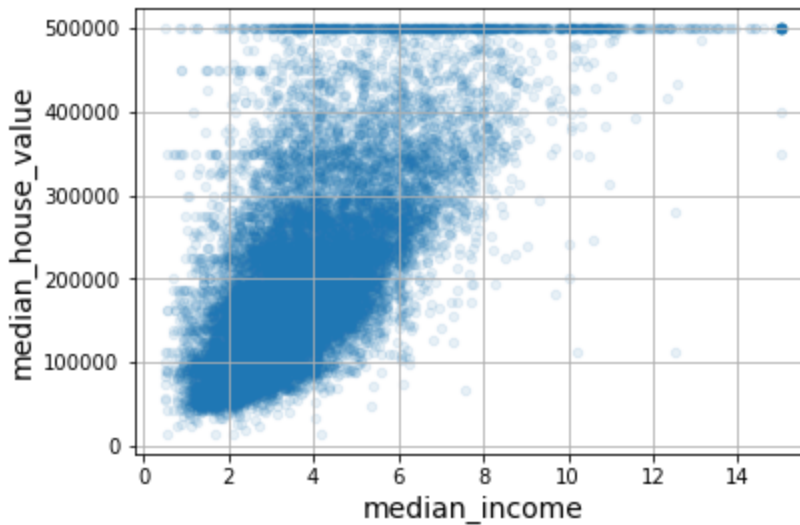
```
median_house_value    1.000000
median_income          0.688380
total_rooms            0.137455
housing_median_age     0.102175
households             0.071426
total_bedrooms         0.054635
population            -0.020153
longitude              -0.050859
latitude               -0.139584
Name: median_house_value, dtype: float64
```

```
from pandas.plotting import scatter_matrix
```

```
attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
save_fig("scatter_matrix_plot") # extra code
plt.show()
```



```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
              alpha=0.1, grid=True)
save_fig("income_vs_house_value_scatterplot") # extra code
plt.show()
```



✓ Thử nghiệm với các kết hợp thuộc tính

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

```
corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income         0.688380
rooms_per_house       0.143663
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population            -0.020153
people_per_house      -0.038224
longitude             -0.050859
latitude              -0.139584
bedrooms_ratio        -0.256397
Name: median_house_value, dtype: float64
```

✓ Chuẩn bị dữ liệu cho thuật toán Machine Learning

Hãy quay lại tập huấn luyện ban đầu và tách các nhãn mục tiêu (lưu ý rằng

`strat_train_set.drop()` tạo ra một bản sao của `strat_train_set` mà không có cột đó, nó không thực sự sửa đổi bản thân `strat_train_set`, trừ khi bạn truyền `inplace=True`):

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

✓ Làm sạch dữ liệu

Trong sách có liệt kê 3 lựa chọn để xử lý các giá trị NaN:

```
housing.dropna(subset=["total_bedrooms"], inplace=True)    # Lựa chọn 1

housing.drop("total_bedrooms", axis=1)                    # Lựa chọn 2

median = housing["total_bedrooms"].median()               # Lựa chọn 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

Đối với mỗi lựa chọn, chúng ta sẽ tạo một bản sao của `housing` và làm việc trên bản sao đó để tránh làm hỏng dữ liệu gốc. Chúng ta cũng sẽ hiển thị kết quả của từng lựa chọn, nhưng lọc trên các hàng ban đầu chứa giá trị NaN.

```
null_rows_idx = housing.isnull().any(axis=1)
housing.loc>null_rows_idx].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	pop
14452	-120.67	40.50	15.0	5343.0	NaN	
18217	-117.96	34.03	35.0	2093.0	NaN	
11889	-118.05	34.04	33.0	1348.0	NaN	
20325	-118.88	34.17	15.0	4260.0	NaN	
14360	-117.87	33.62	8.0	1266.0	NaN	

```
housing_option1 = housing.copy()

housing_option1.dropna(subset=["total_bedrooms"], inplace=True) # option 1

housing_option1.loc>null_rows_idx].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	populati
--	-----------	----------	--------------------	-------------	----------------	----------

```
housing_option2 = housing.copy()

housing_option2.drop("total_bedrooms", axis=1, inplace=True) # option 2
```

```
housing_option2.loc[null_rows_idx].head()
```

	longitude	latitude	housing_median_age	total_rooms	population	household
14452	-120.67	40.50	15.0	5343.0	2503.0	90
18217	-117.96	34.03	35.0	2093.0	1755.0	40
11889	-118.05	34.04	33.0	1348.0	1098.0	25
20325	-118.88	34.17	15.0	4260.0	1701.0	60
14360	-117.87	33.62	8.0	1266.0	375.0	15

```
housing_option3 = housing.copy()
```

```
median = housing["total_bedrooms"].median()
```

```
housing_option3["total_bedrooms"].fillna(median, inplace=True) # option 3
```

```
housing_option3.loc[null_rows_idx].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	pop
14452	-120.67	40.50	15.0	5343.0	434.0	
18217	-117.96	34.03	35.0	2093.0	434.0	
11889	-118.05	34.04	33.0	1348.0	434.0	
20325	-118.88	34.17	15.0	4260.0	434.0	
14360	-117.87	33.62	8.0	1266.0	434.0	

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy="median")
```

Tách riêng các thuộc tính số để sử dụng chiến lược "median" (vì nó không thể tính toán trên các thuộc tính văn bản như ocean_proximity):

```
housing_num = housing.select_dtypes(include=[np.number])
```

```
imputer.fit(housing_num)
```

```
SimpleImputer(strategy='median')
```

```
imputer.statistics_
```



```
array([-118.51 ,  34.26 ,  29.   , 2125.   ,  434.   , 1167.   ,
        408.   ,  3.5385])
```

Kiểm tra xem kết quả này có giống với việc tính toán thủ công trung vị của từng thuộc tính không:

```
housing_num.median().values
```

```
array([-118.51 ,  34.26 ,  29.   , 2125.   ,  434.   , 1167.   ,
        408.   ,  3.5385])
```

Biến đổi tập huấn luyện:

```
X = imputer.transform(housing_num)
```

```
imputer.feature_names_in_
```

```
array(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income'],
      dtype=object)
```

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

```
housing_tr.loc[null_rows_idx].head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	pop
14452	-120.67	40.50	15.0	5343.0	434.0	
18217	-117.96	34.03	35.0	2093.0	434.0	
11889	-118.05	34.04	33.0	1348.0	434.0	
20325	-118.88	34.17	15.0	4260.0	434.0	
14360	-117.87	33.62	8.0	1266.0	434.0	

```
imputer.strategy
```

```
'median'
```

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

```
housing_tr.loc[null_rows_idx].head() # not shown in the book
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	pop
14452	-120.67	40.50	15.0	5343.0	434.0	
18217	-117.96	34.03	35.0	2093.0	434.0	
11889	-118.05	34.04	33.0	1348.0	434.0	
20325	-118.88	34.17	15.0	4260.0	434.0	
14360	-117.87	33.62	8.0	1266.0	434.0	

```
#from sklearn import set_config  
#  
# set_config(transform_output="pandas") # scikit-learn >= 1.2
```

Bây giờ hãy loại bỏ một số điểm ngoại lệ (outliers):

```
from sklearn.ensemble import IsolationForest  
  
isolation_forest = IsolationForest(random_state=42)  
outlier_pred = isolation_forest.fit_predict(X)
```

```
outlier_pred
```

```
array([-1,  1,  1, ...,  1,  1,  1])
```

Nếu bạn muốn loại bỏ các điểm ngoại lệ, bạn sẽ chạy đoạn mã sau:

```
#housing = housing.iloc[outlier_pred == 1]  
#housing_labels = housing_labels.iloc[outlier_pred == 1]
```

✓ Xử lý các thuộc tính văn bản và phân loại

Bây giờ hãy tiền xử lý thuộc tính phân loại đầu vào, `ocean_proximity`:

```
housing_cat = housing[["ocean_proximity"]]  
housing_cat.head(8)
```

	ocean_proximity
13096	NEAR BAY
14973	<1H OCEAN
3785	INLAND
14689	INLAND
20507	NEAR OCEAN
1286	INLAND
18078	<1H OCEAN
4396	NEAR BAY

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

```
housing_cat_encoded[:8]
```

```
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

```
ordinal_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
```

```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

```
housing_cat_1hot
```

```
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
  with 16512 stored elements in Compressed Sparse Row format>
```

Theo mặc định, lớp `OneHotEncoder` trả về một mảng thưa (sparse array), nhưng chúng ta có thể chuyển nó thành mảng dày đặc (dense array) nếu cần bằng cách gọi phương thức `toarray()`:

```
housing_cat_1hot.toarray()

array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Ngoài ra, bạn có thể đặt `sparse_output=False` khi tạo `OneHotEncoder` (lưu ý: siêu tham số `sparse` đã được đổi tên thành `sparse_output` trong Scikit-Learn 1.2):

```
cat_encoder = OneHotEncoder(sparse_output=False)
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
housing_cat_1hot
```

```
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

```
cat_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

```
df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
pd.get_dummies(df_test)
```

	ocean_proximity_INLAND	ocean_proximity_NEAR BAY
0	1	0
1	0	1

```
cat_encoder.transform(df_test)
```

```
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

```
df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN", "ISLAND"]})
pd.get_dummies(df_test_unknown)
```

	ocean_proximity_<2H OCEAN	ocean_proximity_ISLAND
0	1	0
1	0	1

```
cat_encoder.handle_unknown = "ignore"
cat_encoder.transform(df_test_unknown)
```

```
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

```
cat_encoder.feature_names_in_
```

```
array(['ocean_proximity'], dtype=object)
```

```
cat_encoder.get_feature_names_out()
```

```
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)
```

```
df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
                          columns=cat_encoder.get_feature_names_out(),
                          index=df_test_unknown.index)
```

```
df_output
```

	ocean_proximity_<1H OCEAN	ocean_proximity_INLAND	ocean_proximity_ISLAND	ocean_proximity_NEAR BAY	ocean_proximity_NEAR OCEAN
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	1.0	0.0	0.0

✓ Chuẩn hóa đặc trưng (Feature Scaling)

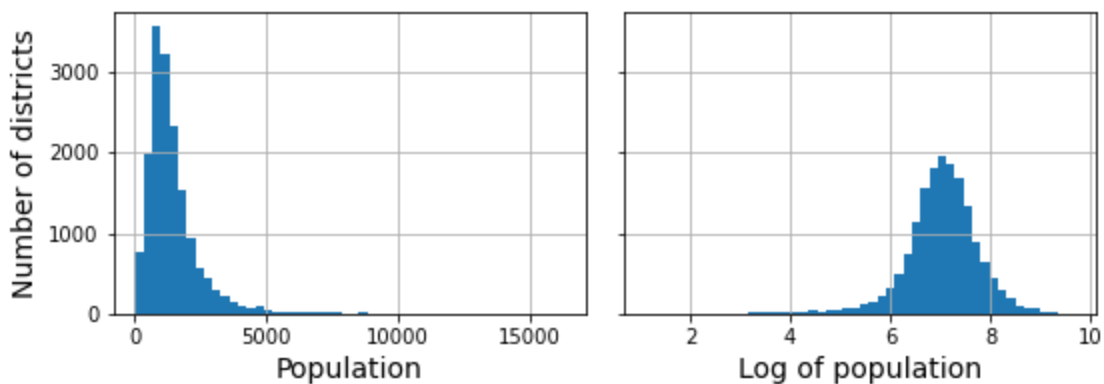
```
from sklearn.preprocessing import MinMaxScaler
```

```
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

```
from sklearn.preprocessing import StandardScaler
```

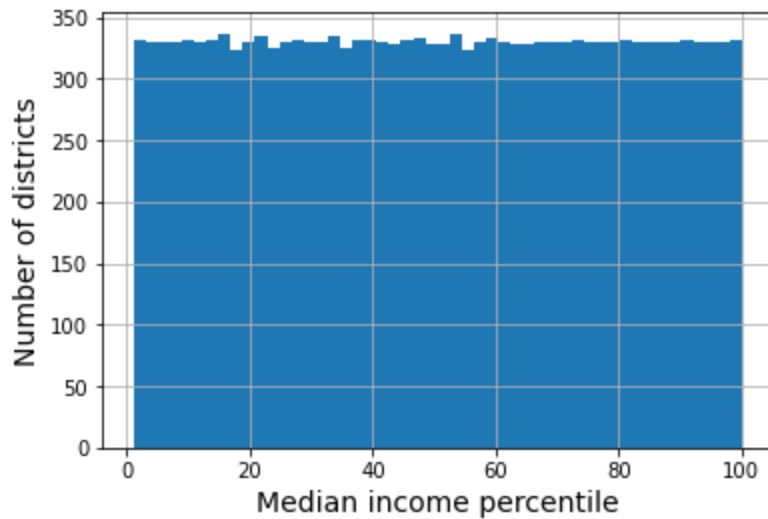
```
std_scaler = StandardScaler()  
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

```
# extra code - this cell generates Figure 2-17  
fig, axs = plt.subplots(1, 2, figsize=(8, 3), sharey=True)  
housing["population"].hist(ax=axs[0], bins=50)  
housing["population"].apply(np.log).hist(ax=axs[1], bins=50)  
axs[0].set_xlabel("Population")  
axs[1].set_xlabel("Log of population")  
axs[0].set_ylabel("Number of districts")  
save_fig("long_tail_plot")  
plt.show()
```



Điều gì xảy ra nếu chúng ta thay thế mỗi giá trị bằng phân vị của nó?

```
# extra code - just shows that we get a uniform distribution  
percentiles = [np.percentile(housing["median_income"], p)  
               for p in range(1, 100)]  
flattened_median_income = pd.cut(housing["median_income"],  
                                 bins=[-np.inf] + percentiles + [np.inf],  
                                 labels=range(1, 100 + 1))  
flattened_median_income.hist(bins=50)  
plt.xlabel("Median income percentile")  
plt.ylabel("Number of districts")  
plt.show()  
# Note: incomes below the 1st percentile are labeled 1, and incomes above the  
# 99th percentile are labeled 100. This is why the distribution below ranges  
# from 1 to 100 (not 0 to 100).
```



```
from sklearn.metrics.pairwise import rbf_kernel
```

```
age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

extra code - this cell generates Figure 2-18

```
ages = np.linspace(housing["housing_median_age"].min(),
                    housing["housing_median_age"].max(),
                    500).reshape(-1, 1)
```

```
gamma1 = 0.1
```

```
gamma2 = 0.03
```

```
rbf1 = rbf_kernel(ages, [[35]], gamma=gamma1)
```

```
rbf2 = rbf_kernel(ages, [[35]], gamma=gamma2)
```

```
fig, ax1 = plt.subplots()
```

```
ax1.set_xlabel("Housing median age")
```

```
ax1.set_ylabel("Number of districts")
```

```
ax1.hist(housing["housing_median_age"], bins=50)
```

```
ax2 = ax1.twinx() # create a twin axis that shares the same x-axis
```

```
color = "blue"
```

```
ax2.plot(ages, rbf1, color=color, label="gamma = 0.10")
```

```
ax2.plot(ages, rbf2, color=color, label="gamma = 0.03", linestyle="--")
```

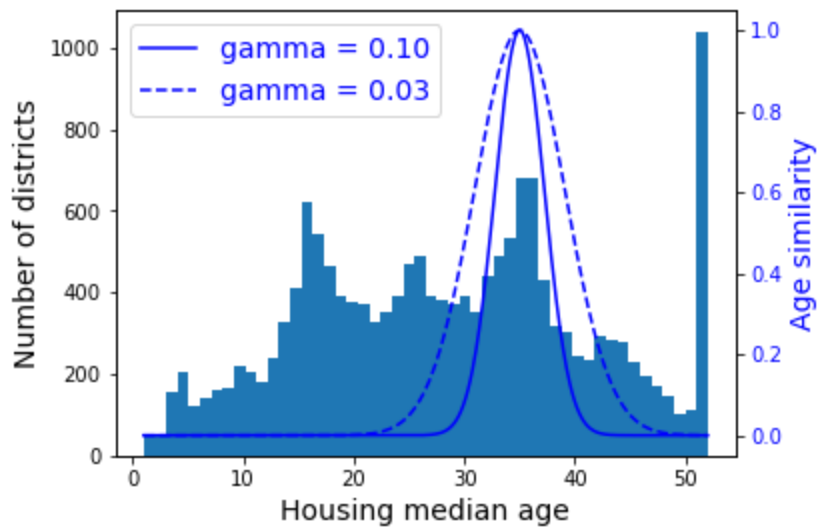
```
ax2.tick_params(axis='y', labelcolor=color)
```

```
ax2.set_ylabel("Age similarity", color=color)
```

```
plt.legend(loc="upper left")
```

```
save_fig("age_similarity_plot")
```

```
plt.show()
```



```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5] # pretend this is new data

scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)
```

predictions

```
array([[131997.15275877],
       [299359.35844434],
       [146023.37185694],
       [138840.33653057],
       [192016.61557639]])
```

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

predictions

```
array([131997.15275877, 299359.35844434, 146023.37185694, 138840.33653057,
       192016.61557639])
```


Các bộ biến đổi tùy chỉnh (Custom Transformers)

Để tạo các bộ biến đổi đơn giản:

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

age_simil_35

```
array([[2.81118530e-13],
       [8.20849986e-02],
       [6.70320046e-01],
       ...,
       [9.55316054e-22],
       [6.70320046e-01],
       [3.03539138e-04]])
```

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords], gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

sf_simil

```
array([[0.999927 ],
       [0.05258419],
       [0.94864161],
       ...,
       [0.00388525],
       [0.05038518],
       [0.99868067]])
```

```
ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
ratio_transformer.transform(np.array([[1., 2.], [3., 4.]])
```

```
array([[0.5 ],
       [0.75]])
```

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted
```

```

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True): # no *args or **kwargs!
        self.with_mean = with_mean

    def fit(self, X, y=None): # y is required even though we don't use it
        X = check_array(X) # checks that X is an array with finite float value
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1] # every estimator stores this in fit()
        return self # always return self!

    def transform(self, X):
        check_is_fitted(self) # looks for learned attributes (with trailing _)
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
            X = X - self.mean_
        return X / self.scale_

```

```

from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, n_init=10,
                               random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self # always return self!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]

```

Cảnh báo:

- Đã có sự thay đổi trong Scikit-Learn 1.3.0 ảnh hưởng đến trình tạo số ngẫu nhiên cho việc khởi tạo `KMeans`. Do đó, kết quả sẽ khác với trong sách nếu bạn sử dụng Scikit-Learn ≥ 1.3 . Đây không phải là vấn đề miễn là bạn không mong đợi kết quả đầu ra giống hệt nhau.
- Xuyên suốt notebook này, khi `n_init` không được thiết lập khi tạo bộ ước lượng `KMeans`, tôi đã đặt nó thành `n_init=10` một cách rõ ràng để tránh cảnh báo về

việc giá trị mặc định sẽ thay đổi từ 10 thành "auto" trong Scikit-Learn 1.4.

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]],
                                           sample_weight=housing_labels)
```

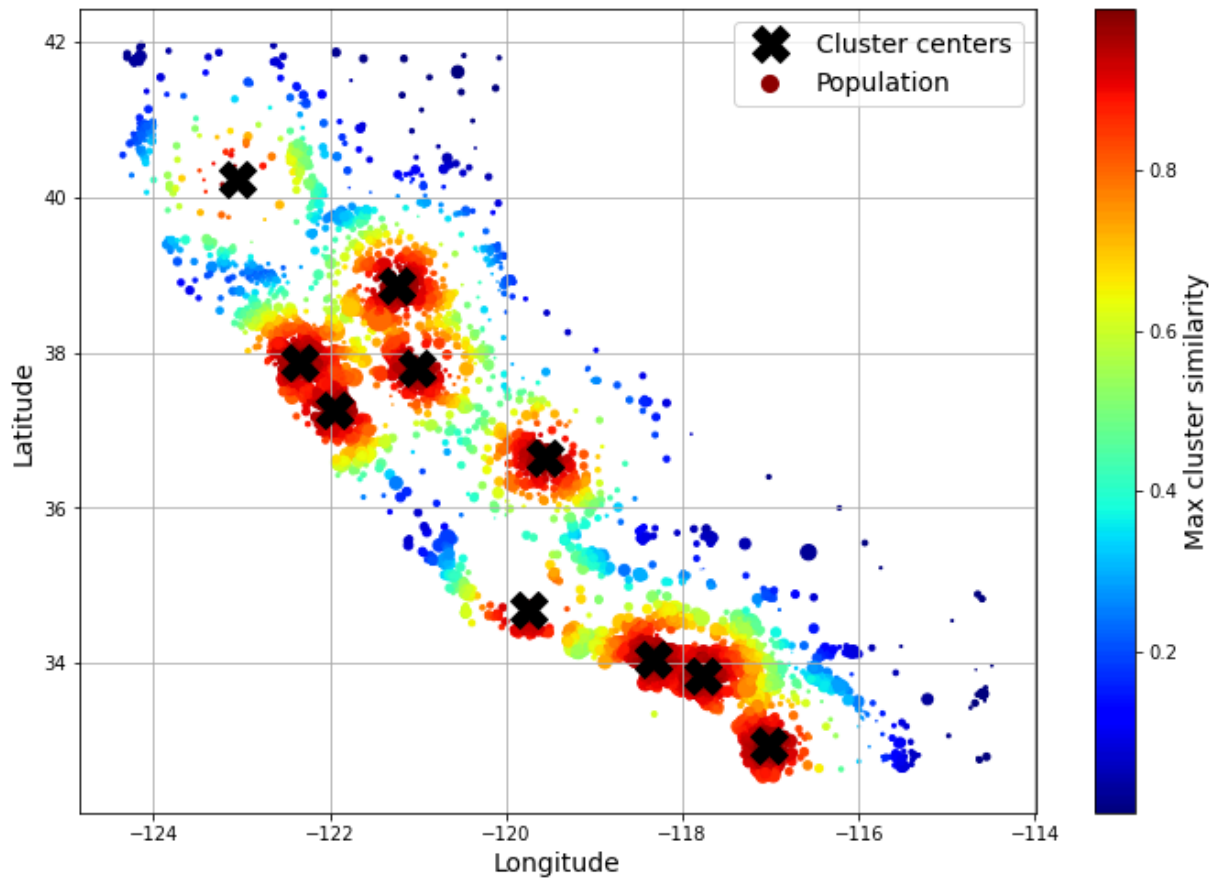
```
similarities[:3].round(2)
```

```
array([[0.  , 0.14, 0.  , 0.  , 0.  , 0.08, 0.  , 0.99, 0.  , 0.6 ],
       [0.63, 0.  , 0.99, 0.  , 0.  , 0.  , 0.04, 0.  , 0.11, 0.  ],
       [0.  , 0.29, 0.  , 0.  , 0.01, 0.44, 0.  , 0.7  , 0.  , 0.3 ]])
```

```
# extra code - this cell generates Figure 2-19
```

```
housing_renamed = housing.rename(columns={
    "latitude": "Latitude", "longitude": "Longitude",
    "population": "Population",
    "median_house_value": "Median house value (usd)"})
housing_renamed["Max cluster similarity"] = similarities.max(axis=1)

housing_renamed.plot(kind="scatter", x="Longitude", y="Latitude", grid=True,
                      s=housing_renamed["Population"] / 100, label="Population",
                      c="Max cluster similarity",
                      cmap="jet", colorbar=True,
                      legend=True, sharex=False, figsize=(10, 7))
plt.plot(cluster_simil.kmeans_.cluster_centers_[0],
         cluster_simil.kmeans_.cluster_centers_[1],
         linestyle="", color="black", marker="X", markersize=20,
         label="Cluster centers")
plt.legend(loc="upper right")
save_fig("district_cluster_plot")
plt.show()
```



✓ Đường ống biến đổi (Transformation Pipelines)

Bây giờ hãy xây dựng một pipeline để tiền xử lý các thuộc tính số:

```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

```
from sklearn import set_config

set_config(display='diagram')
```

```
num_pipeline
```

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median')),  
                 ('standardscaler', StandardScaler())])
```

```
housing_num_prepared = num_pipeline.fit_transform(housing_num)  
housing_num_prepared[:2].round(2)
```

```
array([[ -1.42,  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94],  
       [ 0.6 , -0.7 ,  0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

```

def monkey_patch_get_signature_names_out():
    """Monkey patch some classes which did not handle get_feature_names_out()
    correctly in Scikit-Learn 1.0.*."""
    from inspect import Signature, signature, Parameter
    import pandas as pd
    from sklearn.impute import SimpleImputer
    from sklearn.pipeline import make_pipeline, Pipeline
    from sklearn.preprocessing import FunctionTransformer, StandardScaler

    default_get_feature_names_out = StandardScaler.get_feature_names_out

    if not hasattr(SimpleImputer, "get_feature_names_out"):
        print("Monkey-patching SimpleImputer.get_feature_names_out()")
        SimpleImputer.get_feature_names_out = default_get_feature_names_out

    if not hasattr(FunctionTransformer, "get_feature_names_out"):
        print("Monkey-patching FunctionTransformer.get_feature_names_out()")
        orig_init = FunctionTransformer.__init__
        orig_sig = signature(orig_init)

        def __init__(*args, feature_names_out=None, **kwargs):
            orig_sig.bind(*args, **kwargs)
            orig_init(*args, **kwargs)
            args[0].feature_names_out = feature_names_out

        __init__.__signature__ = Signature(
            list(signature(orig_init).parameters.values()) + [
                Parameter("feature_names_out", Parameter.KEYWORD_ONLY)])

        def get_feature_names_out(self, names=None):
            if callable(self.feature_names_out):
                return self.feature_names_out(self, names)
            assert self.feature_names_out == "one-to-one"
            return default_get_feature_names_out(self, names)

        FunctionTransformer.__init__ = __init__
        FunctionTransformer.get_feature_names_out = get_feature_names_out

    monkey_patch_get_signature_names_out()

```

```

df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)

```

```

df_housing_num_prepared.head(2) # extra code

```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	pop
13096	-1.423037	1.013606	1.861119	0.311912	1.368167	
14973	0.596394	-0.702103	0.907630	-0.308620	-0.435925	

```
num_pipeline.steps
```

```
[('simpleimputer', SimpleImputer(strategy='median')),  
 ('standardscaler', StandardScaler())]
```

```
num_pipeline[1]
```

```
StandardScaler()
```

```
num_pipeline[:-1]
```

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median'))])
```

```
num_pipeline.named_steps["simpleimputer"]
```

```
SimpleImputer(strategy='median')
```

```
num_pipeline.set_params(simpleimputer__strategy="median")
```

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median')),  
 ('standardscaler', StandardScaler())])
```

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",  
               "total_bedrooms", "population", "households", "median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(  
    SimpleImputer(strategy="most_frequent"),  
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([  
    ("num", num_pipeline, num_attribs),  
    ("cat", cat_pipeline, cat_attribs),  
])
```

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(  
    (num_pipeline, make_column_selector(dtype_include=np.number)),
```

```
(cat_pipeline, make_column_selector(dtype_include=object)),
)
```

```
housing_prepared = preprocessing.fit_transform(housing)
```

```
# extra code - shows that we can get a DataFrame out if we want
housing_prepared_fr = pd.DataFrame(
    housing_prepared,
    columns=preprocessing.get_feature_names_out(),
    index=housing.index)
housing_prepared_fr.head(2)
```

	pipeline- 1__longitude	pipeline- 1__latitude	pipeline- 1__housing_median_age	pipeline- 1__total_rooms	pipeline- 1__total_rooms
13096	-1.423037	1.013606	1.861119	0.311912	
14973	0.596394	-0.702103	0.907630	-0.308620	

```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"] # feature names out

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                      StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                           "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
```



```
],
remainder=default_num_pipeline) # one column remaining: housing_median_age
```

```
housing_prepared = preprocessing.fit_transform(housing)
housing_prepared.shape
```

```
(16512, 24)
```

```
preprocessing.get_feature_names_out()
```

```
array(['bedrooms__ratio', 'rooms_per_house__ratio',
      'people_per_house__ratio', 'log__total_bedrooms',
      'log__total_rooms', 'log__population', 'log__households',
      'log__median_income', 'geo__Cluster 0 similarity',
      'geo__Cluster 1 similarity', 'geo__Cluster 2 similarity',
      'geo__Cluster 3 similarity', 'geo__Cluster 4 similarity',
      'geo__Cluster 5 similarity', 'geo__Cluster 6 similarity',
      'geo__Cluster 7 similarity', 'geo__Cluster 8 similarity',
      'geo__Cluster 9 similarity', 'cat__ocean_proximity<1H OCEAN',
      'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
      'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
      'remainder__housing_median_age'], dtype=object)
```

- ✓ Lựa chọn và huấn luyện mô hình
- ✓ Huấn luyện và đánh giá trên tập huấn luyện

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

```
Pipeline(steps=[('columntransformer',
                  ColumnTransformer(remainder=Pipeline(steps=[('simpleimputer',
                                                                    SimpleImputer(strategy='median')),
                                                                    ('standardscaler',
                                                                    StandardScaler())))),
                  transformers=[('bedrooms',
                                Pipeline(steps=[('simpleimputer',
                                                    SimpleImputer(strategy='median')),
                                                    ('functiontransformer',
                                                    FunctionTransformer(feature_names_out=<function ratio_name at 0x1a5...
```

```

        'households',
        'median_income']],
        ('geo',

ClusterSimilarity(random_state=42),

        ['latitude', 'longitude']],
        ('cat',
        Pipeline(steps=

[('simpleimputer',

SimpleImputer(strategy='most_frequent')),

('onehotencoder',

OneHotEncoder(handle_unknown='ignore'))]),

<sklearn.compose._column_transformer.make_column_selector object at
0x1a57e3a00>)])),
        ('linearregression', LinearRegression()))])

```

Hãy thử pipeline tiền xử lý đầy đủ trên một vài thực thể huấn luyện:

```

housing_predictions = lin_reg.predict(housing)
housing_predictions[:5].round(-2) # -2 = rounded to the nearest hundred

array([243700., 372400., 128800., 94400., 328300.])

```

So sánh với các giá trị thực tế:

```

housing_labels.iloc[:5].values

array([458300., 483800., 101700., 96100., 361800.])

```

```

# extra code - computes the error ratios discussed in the book
error_ratios = housing_predictions[:5].round(-2) / housing_labels.iloc[:5].values
print(", ".join([f"{100 * ratio:.1f}%" for ratio in error_ratios]))

```

```

-46.8%, -23.0%, 26.6%, -1.8%, -9.3%

```

Cảnh báo: Trong các phiên bản Scikit-Learn gần đây, bạn phải sử dụng

`root_mean_squared_error(labels, predictions)` để tính RMSE, thay vì `mean_squared_error(labels, predictions, squared=False)`. Khi thử/except sau đây cố gắng import `root_mean_squared_error`, và nếu thất bại nó sẽ tự định nghĩa hàm đó.

```

try:
    from sklearn.metrics import root_mean_squared_error

```

```

except ImportError:
    from sklearn.metrics import mean_squared_error

    def root_mean_squared_error(labels, predictions):
        return mean_squared_error(labels, predictions, squared=False)

lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
lin_rmse

```

```
68687.89176589991
```

```

from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)

Pipeline(steps=[('columntransformer',
                  ColumnTransformer(remainder=Pipeline(steps=[('simpleimputer',
                                                                  SimpleImputer(strategy='median')),
                                                                  ('standardscaler',
                                                                    StandardScaler())])),
                  transformers=[('bedrooms',
                                Pipeline(steps=[('simpleimputer',
                                                    SimpleImputer(strategy='median')),
                                                    ('functiontransformer',
                                                      FunctionTransformer(feature_names_out=<function ratio_name at 0x1a5...
                                                                              ('geo',
                                                                                ClusterSimilarity(random_state=42),
                                                                              ['latitude', 'longitude'])),
                                                                              ('cat',
                                                                                Pipeline(steps=[('simpleimputer',
                                                                                          SimpleImputer(strategy='most_frequent')),
                                                                                          ('onehotencoder',
                                                                                            OneHotEncoder(handle_unknown='ignore'))])),
                                                                              <sklearn.compose._column_transformer.make_column_selector object at
                                                                              0x1a57e3a00>)])),
                                ('decisiontreeregressor',
                                  DecisionTreeRegressor(random_state=42))]))

```

```

housing_predictions = tree_reg.predict(housing)
tree_rmse = root_mean_squared_error(housing_labels, housing_predictions)

```

```
tree_rmse
```

```
0.0
```

✓ Đánh giá tốt hơn bằng cách sử dụng kiểm định chéo (Cross-Validation)

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                               scoring="neg_root_mean_squared_error", cv=10)
```

```
pd.Series(tree_rmses).describe()
```

```
count      10.000000
mean      66868.027288
std       2060.966425
min       63649.536493
25%      65338.078316
50%      66801.953094
75%      68229.934454
max       70094.778246
dtype: float64
```

```
# extra code - computes the error stats for the linear model
lin_rmses = -cross_val_score(lin_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)

pd.Series(lin_rmses).describe()
```

```
count      10.000000
mean      69858.018195
std        4182.205077
min       65397.780144
25%      68070.536263
50%      68619.737842
75%      69810.076342
max       80959.348171
dtype: float64
```

Cảnh báo: ô tiếp theo có thể mất vài phút để chạy:

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))

forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                 scoring="neg_root_mean_squared_error", cv=10)
```

```
pd.Series(forest_rmse).describe()
```

```
count      10.000000
mean      47019.561281
std       1033.957120
min       45458.112527
25%       46464.031184
50%       46967.596354
75%       47325.694987
max       49243.765795
dtype: float64
```

Hãy so sánh RMSE đo được bằng kiểm định chéo ("sai số kiểm định") với RMSE đo được trên tập huấn luyện ("sai số huấn luyện"):

```
forest_reg.fit(housing, housing_labels)
housing_predictions = forest_reg.predict(housing)
forest_rmse = root_mean_squared_error(housing_labels, housing_predictions)
forest_rmse
```

```
17474.619286483998
```

Sai số huấn luyện thấp hơn nhiều so với sai số kiểm định, điều này thường có nghĩa là mô hình đã bị quá khớp (overfit) tập huấn luyện. Một giải thích khả dĩ khác là có sự không khớp giữa dữ liệu huấn luyện và dữ liệu kiểm định, nhưng không phải trường hợp ở đây, vì cả hai đều đến từ cùng một tập dữ liệu mà chúng ta đã xáo trộn và chia làm hai phần.

✓ Tinh chỉnh mô hình của bạn

✓ Tìm kiếm lưới (Grid Search)

Warning: the following cell may take a few minutes to run:

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
```

```

        {'preprocessing__geo__n_clusters': [10, 15],
         'random_forest__max_features': [6, 8, 10]},
    ]
    grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                               scoring='neg_root_mean_squared_error')
    grid_search.fit(housing, housing_labels)

GridSearchCV(cv=3,
             estimator=Pipeline(steps=[('preprocessing',
ColumnTransformer(remainder=Pipeline(steps=[('simpleimputer',
SimpleImputer(strategy='median')),
('standardscaler',
StandardScaler()))]),
transformers=
[('bedrooms',
Pipeline(steps=[('simpleimputer',
SimpleImputer(strategy='median')),
('functiontransformer',
FunctionTransformer(feature_names_out=<f...
<sklearn.compose._column_transformer.make_column_selector object at
0x1a57e3a00>))]),
('random_forest',
RandomForestRegressor(random_state=42))]),
             param_grid=[{'preprocessing__geo__n_clusters': [5, 8, 10],
                           'random_forest__max_features': [4, 6, 8]},
                           {'preprocessing__geo__n_clusters': [10, 15],
                           'random_forest__max_features': [6, 8, 10]}],
             scoring='neg_root_mean_squared_error')

```

Bạn có thể lấy danh sách đầy đủ các siêu tham số có sẵn để điều chỉnh bằng cách xem

```
full_pipeline.get_params().keys():
```

```

# extra code - shows part of the output of get_params().keys()
print(str(full_pipeline.get_params().keys())[:1000] + "...")

```

```
dict_keys(['memory', 'steps', 'verbose', 'preprocessing', 'random_forest', 'prep
```

Kết hợp siêu tham số tốt nhất được tìm thấy:

```
grid_search.best_params_
```

```
{'preprocessing__geo__n_clusters': 15, 'random_forest__max_features': 6}
```

```
grid_search.best_estimator_
```

```
Pipeline(steps=[('preprocessing',  
                  ColumnTransformer(remainder=Pipeline(steps=[('simpleimputer',  
SimpleImputer(strategy='median')),  
                                                                ('standardscaler',  
StandardScaler())])),  
                  transformers=[('bedrooms',  
                                Pipeline(steps=  
[('simpleimputer',  
SimpleImputer(strategy='median')),  
('functiontransformer',  
FunctionTransformer(feature_names_out=<function ratio_name at 0x1a5b6fd...  
ClusterSimilarity(n_clusters=15,  
random_state=42),  
                                                                ['latitude', 'longitude'])),  
('cat',  
Pipeline(steps=  
[('simpleimputer',  
SimpleImputer(strategy='most_frequent')),  
('onehotencoder',  
OneHotEncoder(handle_unknown='ignore'))])),  
                                <sklearn.compose._column_transformer.make_column_selector object at  
0x1a5cdffd0>)])),  
                                ('random_forest',  
RandomForestRegressor(max_features=6, random_state=42)))]])
```

Hãy xem điểm số của mỗi kết hợp siêu tham số được thử nghiệm trong quá trình tìm kiếm lưới:

```
cv_res = pd.DataFrame(grid_search.cv_results_)  
cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)  
  
# extra code - these few lines of code just make the DataFrame look nicer  
cv_res = cv_res[["param_preprocessing__geo__n_clusters",  
                 "param_random_forest__max_features", "split0_test_score",  
                 "split1_test_score", "split2_test_score", "mean_test_score"]]  
score_cols = ["split0", "split1", "split2", "mean_test_rmse"]  
cv_res.columns = ["n_clusters", "max_features"] + score_cols
```

```
cv_res[score_cols] = -cv_res[score_cols].round().astype(np.int64)
```

```
cv_res.head()
```

	n_clusters	max_features	split0	split1	split2	mean_test_rmse
12	15	6	43460	43919	44748	44042
13	15	8	44132	44075	45010	44406
14	15	10	44374	44286	45316	44659
7	10	6	44683	44655	45657	44999
9	10	6	44683	44655	45657	44999

✓ Tìm kiếm ngẫu nhiên (Randomized Search)

```
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingRandomSearchCV
```

Thử 30 ($n_iter \times cv$) tổ hợp siêu tham số ngẫu nhiên:

Warning: the following cell may take a few minutes to run:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distributions = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                       'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

```
RandomizedSearchCV(cv=3,
                   estimator=Pipeline(steps=[('preprocessing',
ColumnTransformer(remainder=Pipeline(steps=[('simpleimputer',
SimpleImputer(strategy='median')),
('standardscaler',
StandardScaler()))]),
transformers=
```



```
[('bedrooms',

Pipeline(steps=[('simpleimputer',

SimpleImputer(strategy='median')),

('functiontransformer',

FunctionTransformer(feature_names_...

<sklearn.compose._column_transformer.make_column_selector object at
0x1a57e3a00>)])),

('random_forest',

RandomForestRegressor(random_state=42))),

param_distributions={'preprocessing_geo_n_clusters':
<scipy.stats._distn_infrastructure.rv_discrete_frozen object at 0x1a4bfc20>,
'random_forest_max_features':
<scipy.stats._distn_infrastructure.rv_discrete_frozen object at 0x1a57c7bb0>},
random_state=42, scoring='neg_root_mean_squared_error')
```

```
# extra code - displays the random search results
cv_res = pd.DataFrame(rnd_search.cv_results_)
cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
cv_res = cv_res[["param_preprocessing_geo_n_clusters",
                 "param_random_forest_max_features", "split0_test_score",
                 "split1_test_score", "split2_test_score", "mean_test_score"]]
cv_res.columns = ["n_clusters", "max_features"] + score_cols
cv_res[score_cols] = -cv_res[score_cols].round().astype(np.int64)
cv_res.head()
```

	n_clusters	max_features	split0	split1	split2	mean_test_rmse
1	45	9	41287	42150	42627	42021
8	32	7	41690	42542	43224	42485
0	41	16	42223	42959	43321	42834
5	42	4	41818	43094	43817	42910
2	23	8	42264	42996	43830	43030

Phần thưởng: cách chọn phân phối lấy mẫu cho một siêu tham số

- `scipy.stats.randint(a, b+1)`: cho các siêu tham số có giá trị *rời rạc* trong khoảng từ a đến b, và tất cả các giá trị trong khoảng đó có khả năng như nhau.
- `scipy.stats.uniform(a, b)`: tương tự, nhưng dành cho các siêu tham số *liên tục*.
- `scipy.stats.geom(1 / scale)`: cho các giá trị rời rạc, khi bạn muốn lấy mẫu đại khái trong một thang đo nhất định. Ví dụ: với scale=1000 hầu hết các mẫu sẽ nằm

trong tầm này, nhưng ~10% mẫu sẽ <100 và ~10% sẽ >2300.

- `scipy.stats.expon(scale)`: đây là dạng liên tục tương đương với `geom`. Chỉ cần đặt `scale` về giá trị có khả năng xảy ra cao nhất.
- `scipy.stats.loguniform(a, b)`: khi bạn gần như không biết thang đo của giá trị siêu tham số tối ưu là gì. Nếu bạn đặt `a=0.01` và `b=100`, thì bạn cũng có khả năng lấy mẫu một giá trị từ 0.01 đến 0.1 tương đương với một giá trị từ 10 đến 100.

Đây là biểu đồ của hàm khối xác suất (cho các biến rời rạc) và hàm mật độ xác suất (cho các biến liên tục) cho `randint()`, `uniform()`, `geom()` và `expon()`:

```
# extra code - plots a few distributions you can use in randomized search

from scipy.stats import randint, uniform, geom, expon

xs1 = np.arange(0, 7 + 1)
randint_distrib = randint(0, 7 + 1).pmf(xs1)

xs2 = np.linspace(0, 7, 500)
uniform_distrib = uniform(0, 7).pdf(xs2)

xs3 = np.arange(0, 7 + 1)
geom_distrib = geom(0.5).pmf(xs3)

xs4 = np.linspace(0, 7, 500)
expon_distrib = expon(scale=1).pdf(xs4)

plt.figure(figsize=(12, 7))

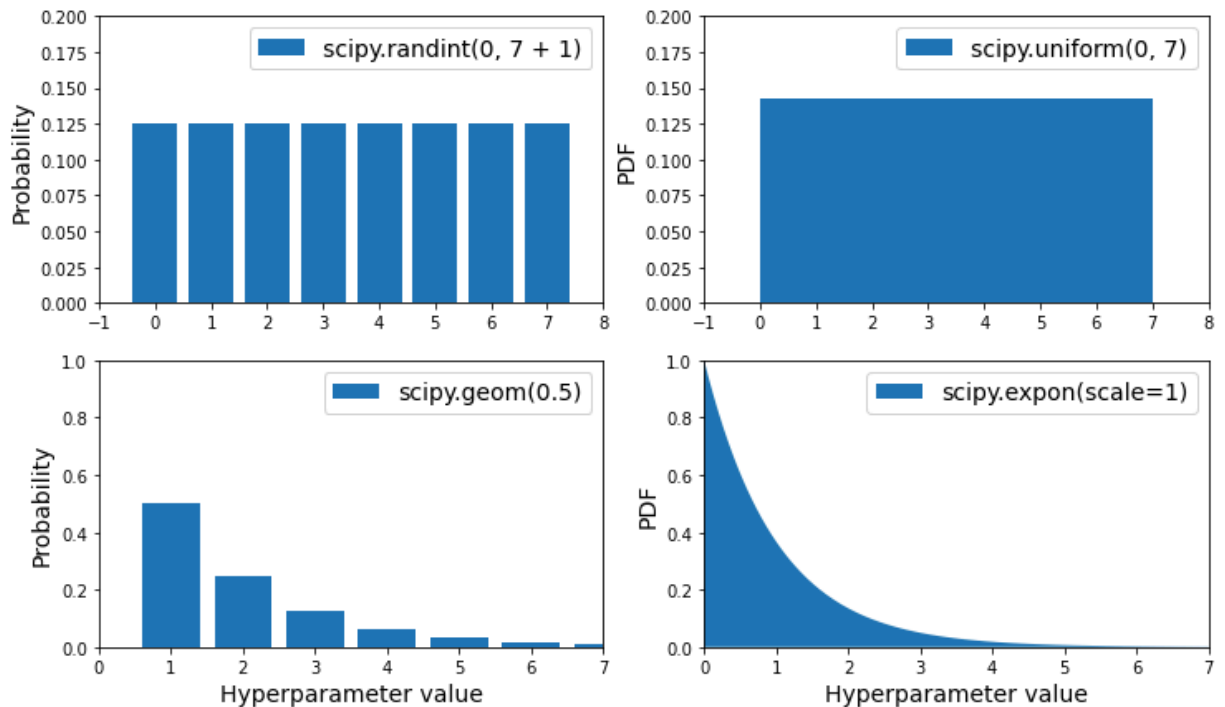
plt.subplot(2, 2, 1)
plt.bar(xs1, randint_distrib, label="scipy.randint(0, 7 + 1)")
plt.ylabel("Probability")
plt.legend()
plt.axis([-1, 8, 0, 0.2])

plt.subplot(2, 2, 2)
plt.fill_between(xs2, uniform_distrib, label="scipy.uniform(0, 7)")
plt.ylabel("PDF")
plt.legend()
plt.axis([-1, 8, 0, 0.2])

plt.subplot(2, 2, 3)
plt.bar(xs3, geom_distrib, label="scipy.geom(0.5)")
plt.xlabel("Hyperparameter value")
plt.ylabel("Probability")
plt.legend()
plt.axis([0, 7, 0, 1])
```

```
plt.subplot(2, 2, 4)
plt.fill_between(xs4, expon_distrib, label="scipy.expon(scale=1)")
plt.xlabel("Hyperparameter value")
plt.ylabel("PDF")
plt.legend()
plt.axis([0, 7, 0, 1])

plt.show()
```



Đây là PDF cho `expon()` và `loguniform()` (cột trái), cũng như PDF của $\log(X)$ (cột phải). Cột bên phải cho thấy sự phân bố của các *thang đo* siêu tham số. Bạn có thể thấy rằng `expon()` ưu tiên các siêu tham số có thang đo mong muốn, với phần đuôi dài hơn về phía các thang đo nhỏ hơn. Nhưng `loguniform()` không ưu tiên bất kỳ thang đo nào, chúng đều có khả năng xảy ra như nhau:

```
# extra code - shows the difference between expon and loguniform

from scipy.stats import loguniform

xs1 = np.linspace(0, 7, 500)
expon_distrib = expon(scale=1).pdf(xs1)

log_xs2 = np.linspace(-5, 3, 500)
log_expon_distrib = np.exp(log_xs2 - np.exp(log_xs2))
```

```

xs3 = np.linspace(0.001, 1000, 500)
loguniform_distrib = loguniform(0.001, 1000).pdf(xs3)

log_xs4 = np.linspace(np.log(0.001), np.log(1000), 500)
log_loguniform_distrib = uniform(np.log(0.001), np.log(1000)).pdf(log_xs4)

plt.figure(figsize=(12, 7))

plt.subplot(2, 2, 1)
plt.fill_between(xs1, expon_distrib,
                 label="scipy.expon(scale=1)")
plt.ylabel("PDF")
plt.legend()
plt.axis([0, 7, 0, 1])

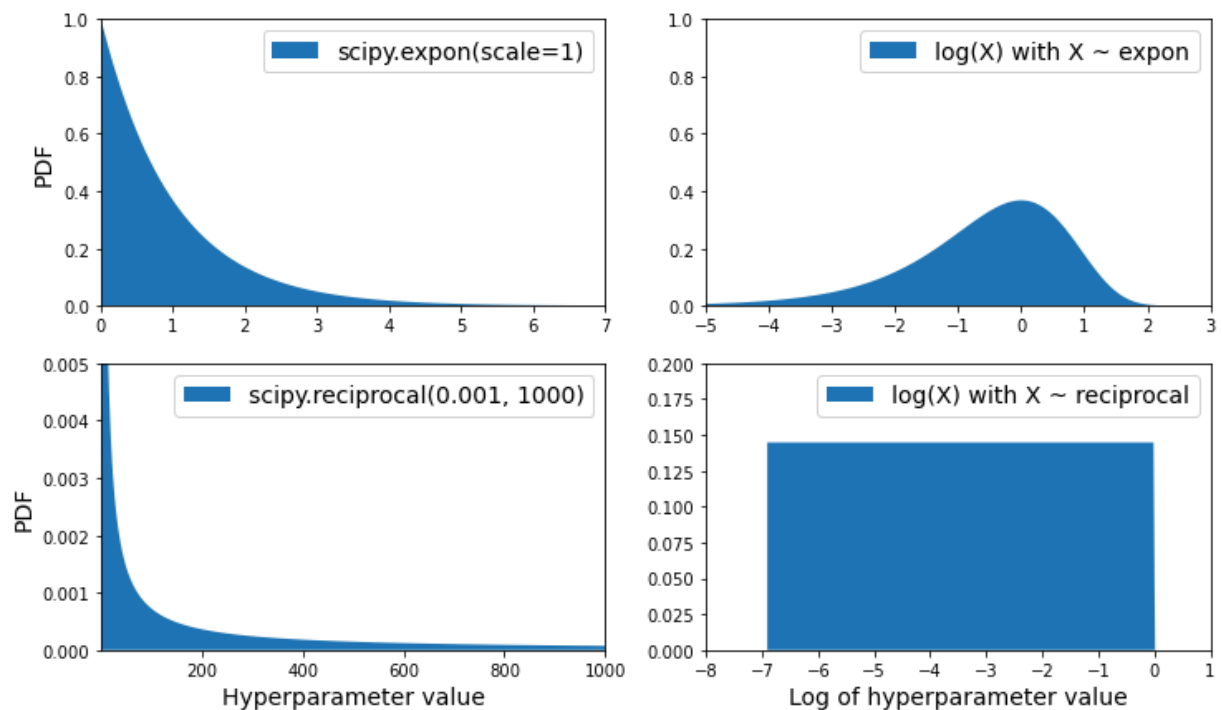
plt.subplot(2, 2, 2)
plt.fill_between(log_xs2, log_expon_distrib,
                 label="log(X) with X ~ expon")
plt.legend()
plt.axis([-5, 3, 0, 1])

plt.subplot(2, 2, 3)
plt.fill_between(xs3, loguniform_distrib,
                 label="scipy.loguniform(0.001, 1000)")
plt.xlabel("Hyperparameter value")
plt.ylabel("PDF")
plt.legend()
plt.axis([0.001, 1000, 0, 0.005])

plt.subplot(2, 2, 4)
plt.fill_between(log_xs4, log_loguniform_distrib,
                 label="log(X) with X ~ loguniform")
plt.xlabel("Log of hyperparameter value")
plt.legend()
plt.axis([-8, 1, 0, 0.2])

plt.show()

```



✓ Phân tích các mô hình tốt nhất và sai số của chúng

```
final_model = rnd_search.best_estimator_ # includes preprocessing
feature_importances = final_model["random_forest"].feature_importances_
feature_importances.round(2)
```

```
array([0.07, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, 0.04, 0.01, 0. ,
       0.01, 0.01, 0.01, 0.01, 0.01, 0. , 0.01, 0.01, 0.01, 0. , 0.01,
       0.01, 0.01, 0.01, 0.01, 0. , 0. , 0.02, 0.01, 0.01, 0.01, 0.02,
       0.01, 0. , 0.02, 0.03, 0.01, 0.01, 0.01, 0.01, 0.01, 0.02, 0.01,
       0.01, 0.02, 0.01, 0.01, 0.01, 0.01, 0.01, 0.02, 0.01, 0. , 0.07,
       0. , 0. , 0. , 0.01])
```

```
sorted(zip(feature_importances,
           final_model["preprocessing"].get_feature_names_out()),
       reverse=True)
```

```
[(0.18694559869103852, 'log__median_income'),
 (0.0748194905715524, 'cat__ocean_proximity_INLAND'),
 (0.06926417748515576, 'bedrooms__ratio'),
 (0.05446998753775219, 'rooms_per_house__ratio'),
 (0.05262301809680712, 'people_per_house__ratio'),
 (0.03819415873915732, 'geo__Cluster 0 similarity'),
 (0.02879263999929514, 'geo__Cluster 28 similarity'),
 (0.023530192521380392, 'geo__Cluster 24 similarity'),
```

```
(0.020544786346378206, 'geo__Cluster 27 similarity'),
(0.019873052631077512, 'geo__Cluster 43 similarity'),
(0.018597511022930273, 'geo__Cluster 34 similarity'),
(0.017409085415656868, 'geo__Cluster 37 similarity'),
(0.015546519677632162, 'geo__Cluster 20 similarity'),
(0.014230331127504292, 'geo__Cluster 17 similarity'),
(0.0141032216204026, 'geo__Cluster 39 similarity'),
(0.014065768027447325, 'geo__Cluster 9 similarity'),
(0.01354220782825315, 'geo__Cluster 4 similarity'),
(0.01348963625822907, 'geo__Cluster 3 similarity'),
(0.01338319626383868, 'geo__Cluster 38 similarity'),
(0.012240533790212824, 'geo__Cluster 31 similarity'),
(0.012089046542256785, 'geo__Cluster 7 similarity'),
(0.01152326329703204, 'geo__Cluster 23 similarity'),
(0.011397459905603558, 'geo__Cluster 40 similarity'),
(0.011383340034016430, 'geo__Cluster 36 similarity')
```